OpenMP Memory Allocators

Lecture notes — HPC 1, Università di Trieste, a.y. 2025/2026
Luca Tornatore, luca.tornatore@inaf.it

The scope of this lecture is to introduce you to the OpenMP memory space and allocators, as high-level tools to manage what memory you get and how you get it.

Disclaimer: all the codes, the accompanying plots, the technical explanations and the concept are from the Author. The text has been re-assembled from sparse materials and transcripts of lectures, with a detailed prompt by Claude, and revised by the Author.

OpenMP Memory Allocators
- Recap
- The "two layers" of OpenMP allocators
- Getting what you want
- Layer 1: memory spaces
- Layer 2: allocators
  - The predefined allocators
- Allocator traits
  - `sync_hint`
  - `alignment`
  - `access`
  - `pool_size`
  - `fallback`
  - `fb_data`
  - `pinned`
  - `partition`
- Contention group vs. parallel team
- Three ways to use allocators
  - 1. API functions: `omp_alloc()` / `omp_free()`
  - 2. The `allocate` directive
  - 3. The `allocate` clause on parallel constructs
- The fallback chain
- The `OMP_ALLOCATOR` environment variable
- Connections to what we have seen before
- Practical caveats
- Summary
  - References

Recap

We've seen in the previous lectures `lecture-notes—Controlling_NUMA_memory_placement` and `lecture-notes—openmp_memory_allocation` how we can manage a proper per-thread memory allocation to get the most appropriate memory for our pruposes.
However, what we have seen assumes that t*here is only one memory type*: typically, the DDRx DRAM. It is not quite the case on modern platforms, where you have many heterogeneous memories: DDRx, HBM, NVDIMM, GPU, ...

OpenMP offers you an abstraction to deal with this heterogeneity without relying on vendor-specific, platform-specific function calls, with a bloat of code within an `#if defined` forest.

The “two layers” of OpenMP allocators

Before getting into the details, let’s consider the whole picture at large.
OpenMP exposes two abstraction layers — memory spaces and allocators below — and three different ways to actually use them.

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                        PREDEFINED MEMORY SPACES                              │
│            (where the storage physically lives — intent, not guarantee)      │
│                                                                              │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐       │
│  │ default  │  │large_cap │  │ high_bw  │  │ low_lat  │  │  const   │       │
│  │ (DDR)    │  │(NVDIMM,  │  │(HBM/     │  │(scratch- │  │(read-    │       │
│  │          │  │ CXL,..)  │  │ MCDRAM)  │  │ pad,..)  │  │ only)    │       │
│  └────┬─────┘  └────┬─────┘  └────┬─────┘  └────┬─────┘  └────┬─────┘       │
│       │             │             │             │             │             │
│       ▼             ▼             ▼             ▼             ▼             │
│                    PREDEFINED ALLOCATORS (ready to use)                      │
│                                                                              │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐       │
│  │ omp_     │  │ omp_     │  │ omp_     │  │ omp_     │  │ omp_     │       │
│  │ default_ │  │large_cap_│  │ high_bw_ │  │ low_lat_ │  │ const_   │       │
│  │ mem_alloc│  │ mem_alloc│  │ mem_alloc│  │ mem_alloc│  │ mem_alloc│       │
│  └──────────┘  └──────────┘  └──────────┘  └──────────┘  └──────────┘       │
│                                                                              │
│             SCOPE-RESTRICTED ALLOCATORS (access limited)                     │
│                                                                              │
│  ┌────────────────┐  ┌────────────────┐  ┌────────────────┐                 │
│  │ omp_cgroup_    │  │ omp_pteam_     │  │ omp_thread_    │                 │
│  │ mem_alloc      │  │ mem_alloc      │  │ mem_alloc      │                 │
│  │ (contention grp)│  │ (parallel team) │  │ (single thread) │                 │
│  └────────────────┘  └────────────────┘  └────────────────┘                 │
│                                                                              │
│             CUSTOM ALLOCATOR (user-defined)                                  │
│             omp_init_allocator(memspace, ntraits, traits[])                  │
│                                                                              │
│  ┌──────────────────────────────────────────────────────────────────┐        │
│  │  Allocator traits (key-value pairs)                              │        │
│  │  ┌──────────┐ ┌──────────┐ ┌──────┐ ┌────────┐ ┌──────┐ ┌────────┐      │        │
│  │  │sync_hint │ │alignment│ │access│ │fallback│ │pinned│ │partition│      │        │
│  │  └──────────┘ └──────────┘ └──────┘ └────────┘ └──────┘ └────────┘      │        │
│  └──────────────────────────────────────────────────────────────────┘        │
│                                                                              │
│             THREE WAYS TO USE AN ALLOCATOR                                   │
│                                                                              │
│  ┌────────────────┐ ┌────────────────┐ ┌──────────────────────────┐         │
│  │ omp_alloc()    │ │ #pragma omp    │ │ allocate(alloc: list)    │         │
│  │ omp_free()     │ │ allocate       │ │ clause on directives     │         │
```

```
│   └──────────────────┘  └──────────────────┘  └──────────────────────────┘          │
└──────────────────────────────────────────────────────────────────────────────────────┘
```

Getting what you want

Up until OpenMP 5.0 the standard had no way of controlling *where* memory physically resides. You could mark variables as `private`, `shared`, `firstprivate` — but those are about *visibility and scope*, not about what piece of hardware the bytes actually sit on. The backing storage always came from whatever `malloc` gave you, or from the thread's stack, and that was it.

In the homogeneous-DRAM era, that was unimportant.
However, in modern HPC node there may be DDR5 main memory, high-bandwidth memory (HBM2e/3e on Sapphire Rapids and on AMD MI300), non-volatile memory of various kinds (Intel Optane, CXL-attached DRAM), GPU device memory — each with very different bandwidth, latency, and capacity. And even within "plain DDR" on a multi-socket node, we have seen how much the NUMA effects are all over the place, as we discussed at length in the shared-memory hardware notes.

How we normally manage to allocate specific memory? Using vendor-specific APIs: `hbw_malloc()` from the memkind library on KNL, `numa_alloc_onnode()` from libnuma, `cudaMalloc()` for GPU memory, and so on. The consequence is a scatter hardware-dependent calls across your code.

OpenMP 5.0 introduced a two-layer abstraction to solve this. The idea is: the programmer says *what properties* are desirable from the memory — "give me something with high bandwidth," "give me something with low latency" — and the runtime figures out how to map that onto whatever hardware is actually present. If the requested hardware is not there, a fallback chain kicks in and a “regular” memory is handled instead. The code compiles and runs everywhere, simply loses the performance benefit on hardware that does not have the special memory.

The wwo layers are:

1. **memory spaces** , which describes the *kind* of storage;
2. **allocators** , they draw from a space with configurable behaviour.

Layer 1: memory spaces

A memory space is an abstract handle. You are declaring an *intent* — "I want storage with these properties" — and the runtime maps that to physical hardware as best it can.
The OpenMP specification defines five predefined memory spaces, and leave the mapping to actual storage to the implementation.

Memory space name	What it means	Typical mapping (when available)
`omp_default_mem_space`	System default	DDR main memory
`omp_large_cap_mem_space`	Large capacity	NVDIMM, CXL-attached, DDR on KNL
`omp_high_bw_mem_space`	High bandwidth	MCDRAM on KNL, HBM2e on Sapphire Rapids

Memory space name	What it means	Typical mapping (when available)
`omp_low_lat_mem_space`	Low latency	Nearest NUMA node, scratchpad on accelerators
`omp_const_mem_space`	Constant values; writing is undefined	`__constant__` on GPUs, read-only pages on CPUs

Some notes worth stressing.

a) these are **hints, not guarantees.** On Intel's oneAPI compiler, for instance, if you ask for `omp_high_bw_mem_space` and the machine has no HBM, the runtime silently maps it to `omp_default_mem_space`. Same for the other non-default spaces — they all collapse to default when the corresponding hardware is not present: never just assume the placement happened.

b) the distinction between "large capacity" and "high bandwidth" is relative to the system. On a KNL node in flat mode, MCDRAM (16 GB, around 450 GB/s) is the high-bandwidth space and DDR4 (96 GB, around 90 GB/s) is the large-capacity space. On a different machine the very same names might point to entirely different physical resources.

c) when using `omp_const_mem_space` you must *not* write to memory allocated there after initialisation. The initialisation can happen through `firstprivate` (which copies at region entry) or through compile-time constant expressions for static variables. On GPUs this maps to constant memory, which has its own dedicated cache and broadcast-read semantics; on CPUs it might just be regular memory with no special treatment at all.

Layer 2: allocators

An allocator is an object that draws memory from a specific memory space, with configurable behaviour controlled by a set of **traits** — key-value pairs that tell the allocator things like "align to 64 bytes" or "abort if allocation fails." There are eight predefined allocators (one per memory space, plus three scope-restricted ones), and you can create your own with `omp_init_allocator()`.

The predefined allocators

Allocator	Memory space	Non-default traits
`omp_default_mem_alloc`	`omp_default_mem_space`	fallback = `null_fb`
`omp_large_cap_mem_alloc`	`omp_large_cap_mem_space`	(all defaults)
`omp_high_bw_mem_alloc`	`omp_high_bw_mem_space`	(all defaults)
`omp_low_lat_mem_alloc`	`omp_low_lat_mem_space`	(all defaults)
`omp_const_mem_alloc`	`omp_const_mem_space`	(all defaults)
`omp_cgroup_mem_alloc`	impl-defined	access = `cgroup`
`omp_pteam_mem_alloc`	impl-defined	access = `pteam`
`omp_thread_mem_alloc`	impl-defined	access = `thread`

The first five are self-explaining: one allocator per space, default traits; you grab one and use it directly when all you need is "put this data in HBM" or "put this data in low-latency memory."

The last three — the scope-restricted allocators — are more interesting, and we'll go more in details in few paragraphs.

Allocator traits

When you create a custom allocator with `omp_init_allocator()`, you pass an array of trait structures (type `omp_alloctrait_t`) that configure its behaviour. Here is the full list.

`sync_hint`

Tells the runtime how much contention to expect on this allocator. Values:

- `contended` — many threads will allocate simultaneously; the runtime may use a scalable allocator with per-thread arenas.
- `uncontended` — few threads, low contention.
- `serialized` — only one thread at a time will use this allocator. **If you violate this and call from multiple threads, behaviour is unspecified.** Not a performance hint. A correctness contract.
- `private` — always the same thread. Same caveat, even stronger: sequential calls from *different* threads are also undefined.

Default: `contended`.

`alignment`

Minimum byte alignment for all allocations from this allocator. Default is 1, which in practice means whatever `malloc` provides (typically 16 bytes on x86-64). For SIMD work you would set this to 32 (AVX/AVX2) or 64 (AVX-512). If you are doing a blocked matrix multiply and want your sub-blocks aligned for vectorised loads, a custom allocator with `alignment = 64` is cleaner than sprinkling `aligned_alloc()` calls around.

`access`

Who can see the allocated memory:

- `all` — any thread on the device (default)
- `cgroup` — all threads in the same contention group
- `pteam` — all threads in the same parallel team
- `thread` — only the allocating thread

What exactly "contention group" and "parallel team" mean is the subject of a dedicated section below — for now, just note that they refer to different scopes in the thread hierarchy, and the difference only becomes visible with nested parallelism.

Note: this `access` trait is more understandable when considered in the frame of GPUs.

`pool_size`

Maximum total bytes this allocator can hand out. Default is implementation-defined (effectively unlimited). Useful for bounding how much HBM you consume: create an HBM allocator with `pool_size` matching the HBM capacity and a fallback to DDR, so overflow goes to slow memory rather than crashing.

`fallback`

What happens when an allocation fails. In practice, this is the most operationally important trait:

- `default_mem_fb` — retry in `omp_default_mem_space` (default for most predefined allocators)
- `null_fb` — return NULL (default for `omp_default_mem_alloc` specifically)
- `abort_fb` — terminate the program
- `allocator_fb` — delegate to another allocator specified by `fb_data`

`fb_data`

Which allocator to fall back to when `fallback = allocator_fb`. This is how you build explicit chains: "try HBM, then large-capacity, then default, then give up." We'll see an example shortly.

`pinned`

If `true`, allocated memory will not be paged out or migrated by the OS. This matters for InfiniBand RDMA (the NIC needs stable physical addresses for zero-copy), for GPU DMA (same reason), and for any latency-sensitive code where you cannot tolerate page faults in the inner loop. Default: `false`.

Remind: GCC's libgomp also provides a non-standard extension, `ompx_gnu_pinned_mem_alloc`, which is a predefined allocator with `pinned = true` already set. Handy if you just want pinned DDR without creating a custom allocator.

`partition`

How to distribute the allocation across storage resources in the memory space:

- `environment` — let the OS decide (default)
- `nearest` — allocate on the resource closest to the requesting thread (first-touch NUMA behaviour)
- `blocked` — split into roughly equal blocks, one per storage resource
- `interleaved` — round-robin across resources (like `numactl --interleave`)

This maps directly onto the NUMA placement strategies from the shared-memory hardware notes: `nearest` is what you want when each thread works on its own slice; `interleaved` spreads bandwidth when all threads touch all data.

However the `partition` trait is largely unimplemented for host allocators in most runtimes as of 2025. Both GCC's libgomp and LLVM's libomp essentially ignore it. For GPU target regions it might eventually matter. But on the host, today, if you need NUMA interleaving you still reach for `numactl`. The spec is ahead of the implementations.

Contention group vs. parallel team

This is one of those distinctions that looks entirely pointless until you have nested parallelism — and then suddenly it is the difference between your code working and producing garbage. So let me draw the situation first, and then explain.

```
┌ - - -
```

When does this actually matter? Suppose thread T1.1 — a member of inner team B — allocates some memory using `omp_pteam_mem_alloc`. That memory is visible only to T1.0, T1.1, and T1.2, the three threads in team B. Thread T0 from the outer team? Cannot see it. Thread T2.1 from team C? Cannot see it either. Trying to access it from outside team B is unspecified behaviour.

If instead T1.1 allocates with `omp_cgroup_mem_alloc`, the memory is visible to *every* thread in the contention group — T0, T1, T2, T3, T1.0–T1.2, T2.0–T2.2, all of them. It is the broadest sharing scope short of `access = all` (which, on a multi-device system, extends to all threads on the entire device).

Without nested parallelism — which is the overwhelmingly common case on the host — the contention group is just {initial thread + the one and only parallel team}, and `omp_cgroup_mem_alloc` and `omp_pteam_mem_alloc` behave identically. The distinction only shows up when you nest.

The place where this really matters is GPU offloading. In a `target teams distribute parallel for` construct you have two levels of parallelism: the outer level is a *league* of teams (one per SM or CU), and the inner level is a parallel team within each league member. Memory allocated with `omp_pteam_mem_alloc` maps to GPU shared memory — the per-SM scratchpad, fast and small, local to the team. If you mistakenly used `omp_cgroup_mem_alloc` instead, that memory would need to be visible across *all* teams on the device; on a GPU, that forces it into global memory, and you lose the entire point of the specialised allocator.

Remind: `omp_pteam_mem_alloc` inside a `teams distribute` region is the portable equivalent of CUDA `__shared__` memory. That single fact is probably the most practically useful thing about the scope-restricted allocators for anyone doing GPU work.

Three ways to use allocators

There are three mechanisms, each working at a different level of granularity. Pick the one that fits.

1. API functions: `omp_alloc()` / `omp_free()`

The most explicit approach — a drop-in replacement for `malloc` / `free`, plus an allocator argument:

```
#include <omp.h>
#include <stdio.h>

int main() {
    int N = 1024 * 1024;

    // Using a predefined allocator directly:
    double *hbw_array = (double*)omp_alloc(
        N * sizeof(double),
        omp_high_bw_mem_alloc
    );

    if (hbw_array == NULL) {
        // default fallback for omp_high_bw_mem_alloc is default_mem_fb,
        // so if we get NULL it means even DDR failed. Truly out of memory.
        fprintf(stderr, "Allocation failed\n");
        return 1;
    }
```

```
    // ... compute on hbw_array ...

    omp_free(hbw_array, omp_high_bw_mem_alloc);
    return 0;
}
```

For a **custom allocator** with specific traits — say, 64-byte aligned and pinned in HBM:

```
#include <omp.h>

int main() {
    int N = 1024 * 1024;

    omp_alloctrait_t traits[2];
    traits[0].key   = omp_atk_alignment;
    traits[0].value = 64;
    traits[1].key   = omp_atk_pinned;
    traits[1].value = omp_atv_true;

    omp_allocator_handle_t my_alloc =
        omp_init_allocator(omp_high_bw_mem_space, 2, traits);

    if (my_alloc == omp_null_allocator) {
        fprintf(stderr, "Failed to create allocator\n");
        return 1;
    }

    double *buf = (double*)omp_alloc(N * sizeof(double), my_alloc);
    // ... work with buf ...
    omp_free(buf, my_alloc);

    omp_destroy_allocator(my_alloc);
    return 0;
}
```

OpenMP 5.1 also added `omp_aligned_alloc()`, which takes an explicit per-call alignment overriding the allocator's `alignment` trait:

```
double *aligned_buf = (double*)omp_aligned_alloc(
    64,                         // alignment in bytes
    N * sizeof(double),         // size
    omp_high_bw_mem_alloc       // allocator
);
```

Remind: you must free memory with `omp_free()`, not with `free()`. The allocator may not use the system heap internally, so `free()` on a pointer from `omp_alloc()` is undefined behaviour. Easy to forget, annoying to debug.

2. The `allocate` directive

This controls the allocation of *declared variables*. You annotate the variable and the runtime places it in the right memory; the code that *uses* the variable stays untouched:

```
void compute(int N) {
    double v1[N], v2[N];        // hot arrays — want these in HBM
    double v3[64 * N];          // cold, large — DDR is fine

    #pragma omp allocate(v1, v2) allocator(omp_high_bw_mem_alloc)
    #pragma omp allocate(v3)     allocator(omp_large_cap_mem_alloc)

    // v1 and v2 sit in HBM (if present), v3 in large-capacity storage.
    // If HBM is not there, fallback kicks in silently.

    init(v1, v2, N, v3, 64*N);
    saxpy(v1, v2, 5.0, N);
    // ...

    // Deallocation is automatic when the variables go out of scope.
}
```

The directive must appear in the same scope as the declarations, and must follow them. For `static` or module-scope variables (Fortran `SAVE`, C `static`), only predefined allocators are allowed — no custom allocators, because lifetime management gets complicated.

3. The `allocate` clause on parallel constructs

This controls where *private copies* of variables are placed. This is where the GPU story becomes interesting:

```
void gather_work(int N) {
    double v5[N], v6[N];

    #pragma omp task private(v5, v6) \
            allocate(omp_low_lat_mem_alloc: v5, v6)
    {
        // Each task's private copies of v5, v6 live in low-latency memory.
        my_gather(v5, v6, N);
    }
}
```

And the GPU case — the portable equivalent of CUDA `__shared__` memory:

```
void gpu_work(double *v3, double *v4, int N) {
    #pragma omp target map(to: v3[0:N], v4[0:N]) map(from: v3[0:N])
    #pragma omp teams distribute
    for (int i = 0; i < N; i += BLOCK) {
        double local[BLOCK];

        #pragma omp parallel private(local) \
                allocate(omp_pteam_mem_alloc: local)
```

```
        {
            // On a GPU, 'local' lives in shared memory (per-SM scratchpad).
            // On a CPU, it is just a thread-local buffer. Nothing special.
            // Same source code, different hardware, right thing happens.
        }
    }
}
```

The `allocate` clause applies only to private copies created by data-sharing clauses (`private`, `firstprivate`, `lastprivate`). It does not touch the original variable. Also: for `task` and `taskloop` constructs, you cannot use allocators with `access = thread`, because tasks can migrate between threads and the memory would become inaccessible.

The fallback chain

The fallback mechanism is what makes the whole system portable, and it is what students tend to overlook. Walk through an example with me.

```
double *buf = omp_alloc(N * sizeof(double), omp_high_bw_mem_alloc);
```

You run this on a machine without HBM. What happens?

The default fallback for `omp_high_bw_mem_alloc` is `default_mem_fb`, so the runtime retries the allocation in `omp_default_mem_space` — regular DDR. The code does not crash; it silently uses slower memory. This is the portability contract. Same binary on both HBM and non-HBM machines; you just lose the performance benefit.

But `omp_default_mem_alloc` has fallback `null_fb`. If the retry in DDR *also* fails (genuinely out of memory), it returns NULL. The chain terminates; no infinite loop.

You can build explicit chains:

```
// "Try HBM → fall back to large-capacity → fall back to default → NULL"
omp_alloctrait_t hbm_traits[2];
hbm_traits[0].key   = omp_atk_fallback;
hbm_traits[0].value = omp_atv_allocator_fb;
hbm_traits[1].key   = omp_atk_fb_data;
hbm_traits[1].value = (omp_uintptr_t)omp_large_cap_mem_alloc;

omp_allocator_handle_t tiered_alloc =
    omp_init_allocator(omp_high_bw_mem_space, 2, hbm_traits);

// tiered_alloc tries:
//   1. HBM (high_bw_mem_space)
//   2. large_cap_mem_alloc (via fb_data)     → tries large_cap_mem_space
//      └─ fallback: default_mem_fb           → tries default_mem_space
//         └─ fallback: null_fb               → returns NULL
```

This tiered pattern is the right approach for production codes. You express your performance preference — "I really want HBM for this" — without sacrificing correctness on machines that do not have it.

The `OMP_ALLOCATOR` environment variable

You can set the default allocator for an entire program without touching source code:

```
export OMP_ALLOCATOR=omp_high_bw_mem_alloc
```

This changes the `def-allocator-var` ICV, used by every allocation call, directive, and clause that does not specify an allocator explicitly. Coarse knob, good for quick experiments. "Does my code benefit from HBM at all?" Set the variable, run, measure. No recompilation needed.

GCC's libgomp supports a more expressive syntax:

```
export OMP_ALLOCATOR="omp_low_lat_mem_space:pinned=true,partition=nearest"
```

This creates a custom allocator at startup with those traits and makes it the default. Handy for experimentation; for production you want the traits in the source code where they are documented and reviewable.

Connections to what we have seen before

The allocator machinery is not isolated; it ties into several topics from earlier lectures.

NUMA placement (shared-memory hardware notes): the `partition` trait with `nearest` and `interleaved` is the OpenMP-portable equivalent of `numactl --localalloc` and `numactl --interleave`. Combined with `proc_bind(spread)` for thread pinning, you get the same first-touch or round-robin behaviour — expressed within the standard rather than through a Linux-specific tool. Caveat: current implementations mostly ignore `partition` for host allocations, so in practice you may still need `numactl` today.

Cache alignment (basic optimisation notes): the `alignment` trait replaces the whole `_mm_malloc()` / `aligned_alloc()` / `posix_memalign()` mess. One allocator with `alignment = 64`, use `omp_alloc()` everywhere. This also works with the `allocate` directive for stack VLAs, which is something you genuinely cannot do with `posix_memalign()`.

Pinned memory (MPI / GPU transfers): InfiniBand RDMA and GPU DMA engines need the physical address of a buffer to stay fixed during the transfer. The `pinned` trait gives you this portably; it is the OpenMP equivalent of `cudaMallocHost()` or `ibv_reg_mr()`. Whether the runtime does it via `mlock()` or something else is not your concern.

Practical caveats

Compiler support is uneven. GCC 15 supports most of OpenMP 5.0/5.1/5.2, and the basic `omp_alloc`/`omp_free` plus the `allocate` directive work fine for host code. Clang/LLVM has been adding allocator support progressively. But allocators inside `target` regions are more fragile — the interaction with device offloading is still under active development in both compilers. Before building production code around this, check what your compiler version actually implements.

"Implementation defined" does a lot of work in this part of the standard. `omp_high_bw_mem_space` *represents* high-bandwidth storage, but if the runtime cannot discover HBM on your system, it maps silently to default. On GCC's libgomp the mapping goes through `memkind` or system-level NUMA APIs when available; without them, everything is `malloc`. You cannot just trust the allocator name — you need to verify the actual mapping on your machine.

Thread-safety depends on `sync_hint`. I know I said this already, but it bears repeating because it is a correctness trap, not a performance hint. If you set `sync_hint` to `serialized` or `private` and then call the allocator from multiple threads, the runtime is free to skip all locking — because you promised single-thread access. Concurrent calls *will* corrupt internal state. Not "might." Will.

Summary

What you want	How	Typical use case
Allocate in HBM	`omp_alloc(size, omp_high_bw_mem_alloc)`	Hot arrays, bandwidth-bound kernels
Aligned allocation	Custom allocator with `omp_atk_alignment`	SIMD inner loops
Pinned memory	Custom allocator with `omp_atk_pinned`	RDMA, GPU DMA transfers
GPU shared memory	`allocate(omp_pteam_mem_alloc: var)` clause	Per-team scratchpad in `target teams`
NUMA-local placement	Custom allocator with `partition = nearest`	Per-thread arrays on multi-socket nodes
Tiered fallback	Chain via `fallback = allocator_fb` / `fb_data`	Portable across HBM and non-HBM systems
Quick experiment	`OMP_ALLOCATOR` environment variable	No recompilation needed

The mental model in one sentence: memory spaces say *what kind of storage you want*; allocators say *how to get it*; and the three usage mechanisms — API calls, directive, clause — let you apply the allocator at the granularity that fits your code.

References

- OpenMP 5.0 Specification, §2.11 (Memory Management): openmp.org/spec-html/5.0
- OpenMP 5.1 Specification, §2.13 (Memory Management): openmp.org/spec-html/5.1
- OpenMP Examples, Chapter 11.2 (Memory Allocators): passlab.github.io/Examples
- A. Roussel et al., "Preliminary Experience with OpenMP Memory Management" (2020), hal.science/hal-02902719
- GCC libgomp `OMP_ALLOCATOR` documentation: gcc.gnu.org/onlinedocs/libgomp
- Intel oneAPI OpenMP Memory Spaces and Allocators: intel.com